Federated Averaging for Multi-Class Network Intrusion Classification: Theoretical Foundations, Empirical Evaluation, and Resilience to Label Poisoning on the CIC–UNSW-NB15 Dataset

Jay Mehta

Independent Research / Academic Project

***Abstract*—**Federated learning (FL) enables multiple institutions to jointly train machine learning models without exchanging raw data, a property that is especially desirable for network security analytics where flow telemetry may contain sensitive operational information. This paper presents a comprehensive theoretical and empirical study of Federated Averaging (FedAvg) applied to multi-class intrusion / traffic classification on the CIC–UNSW-NB15 augmented dataset. We formalize the FL objective, the FedAvg update rule, local empirical risk minimization with multilayer perceptrons (MLPs), and a single-client label-poisoning threat model. We then conduct a 64-configuration experimental sweep over client counts, global communication rounds, local epochs, and poison intensities. Empirically, longer training improves clean accuracy (best clean accuracy ≈ 0.924), while targeted poisoning is most harmful when a single client dominates training (worst accuracy ≈ 0.856). With multiple clients, FedAvg dilutes one poisoned shard and accuracy typically remains above 0.90. Confusion-matrix and one-vs-rest TP/FP/FN/TN analyses show that majority-class targeted poisoning systematically increases false positives for class 0 and reduces recall for several minority attack categories—an effect that can be understated by overall accuracy alone under severe class imbalance. Interactive dashboard: https://fl-research.vercel.app.

***Keywords*—**Federated learning, Federated Averaging (FedAvg), intrusion detection systems, network traffic classification, data poisoning, label flipping, CIC–UNSW-NB15, multilayer perceptron, confusion matrix, class imbalance

I. INTRODUCTION

A. Motivation

Network intrusion detection and traffic classification increasingly rely on supervised learning over high-dimensional flow features. In operational settings, however, the organizations that collect such telemetry—enterprises, internet service providers, campus networks, or industrial control environments—are often unwilling or legally unable to centralize raw records. Privacy regulations, competitive sensitivity, and breach-risk all discourage naive data pooling.

Federated learning offers an alternative: each participant trains on local data and shares only model parameters (or gradients), which a coordinating server aggregates into a global model. For intrusion detection, this paradigm suggests a path toward collaborative detection quality without collaborative data warehouses.

B. Problem Statement

We study the following concrete problem. Given a tabular multi-class dataset with feature vectors $x \in \blacksquare^d$ and labels $y \in \{0,\ldots,C-1\}$, simulate N clients holding disjoint training shards and learn a shared classifier f_θ via FedAvg. We then ask:

1. How do N, the number of global rounds R, and local epochs E affect test accuracy and F1-score?
2. How does label poisoning on a single client, with fraction f of labels corrupted, change global performance?
3. Does poisoning alter *which* classes are confused, beyond aggregate metrics?

C. Scope and Simulation Nature

All experiments in this repository run on a single machine. “Clients” are logical partitions of a shuffled training set rather than physically distributed hosts. The simulation preserves the algorithmic structure of FL (local optimization + server aggregation) while abstracting away network latency, dropouts, and systems engineering. This is appropriate for controlled scientific comparison of learning dynamics and poisoning effects, but it is not a full systems deployment study.

D. Contributions

1. A self-contained theoretical presentation of FedAvg for multi-class classification, including the global objective, local surrogate losses, weighted aggregation, and a formal label-poisoning adversary.

2. An end-to-end reproducible pipeline on CIC–UNSW-NB15 augmented data: cleaning, stratified splitting, scaling, IID sharding, MLP training, and evaluation with confusion matrices and per-class TP/FP/FN/TN.
3. A 64-run experimental grid quantifying interactions among clients, rounds, local epochs, and poison settings, with best-case clean accuracy near 0.924 and worst-case poisoned accuracy near 0.856.
4. Evidence that majority-class targeted poisoning induces a "predict class 0 when uncertain" bias that is most severe under single-client training and largely diluted under multi-client FedAvg.
5. Public interactive visualizations of the full experimental record.

E. Paper Organization

Section II reviews FL, IDS learning, and poisoning. Section III develops the mathematical framework. Section IV describes the system design. Section V details the experimental protocol. Sections VI–VII present and interpret results. Section VIII concludes.

II. BACKGROUND AND RELATED WORK

A. Centralized Learning for Intrusion Detection

Classical machine learning for IDS ranges from shallow models (decision trees, SVMs, random forests) to deep neural networks trained on packet or flow features. Centralized training assumes a single data lake. While often accurate in benchmarks, this assumption conflicts with multi-organization collaboration. Moreover, network data are characteristically *imbalanced*: benign or majority classes dominate, so accuracy can be misleading unless accompanied by per-class analysis. Cost-sensitive learning, resampling, and anomaly-detection formulations are common responses; in this work we retain a standard multi-class MLP and instead emphasize diagnostic metrics.

B. Federated Learning Basics

In cross-silo or cross-device FL, a server coordinates rounds of local training and aggregation. FedAvg is the canonical algorithm: clients run multiple local SGD/Adam steps and return updated weights; the server averages them, typically weighted by local dataset sizes. The literature has since developed methods for statistical heterogeneity (FedProx, SCAFFOLD), communication compression, personalization, secure aggregation, and robust aggregation against Byzantine clients. Understanding vanilla FedAvg remains foundational because it is still the default baseline in many applied FL systems.

A central systems insight of FedAvg is that increasing local computation (more local epochs) can reduce the number of communication rounds needed to reach a target accuracy when client data are not too heterogeneous. That trade-off is directly relevant to IDS collaborations spanning administrative domains with limited bandwidth or intermittent connectivity.

C. FL for Security Analytics

Prior work has explored FL for IoT and network intrusion detection to keep telemetry local while improving collective detectors. Our contribution is not a new FL optimizer; rather, we provide a carefully instrumented FedAvg case study with poisoning analysis and confusion-level diagnostics on a widely used tabular intrusion dataset, accompanied by public artifacts.

D. Poisoning and Byzantine Behavior

Data poisoning injects corrupted training examples to degrade or redirect a learned model. In FL, a malicious client can poison labels or features, or upload crafted weights (model poisoning / backdoor insertion). Defenses include robust aggregators (coordinate-wise median, trimmed mean, Krum), anomaly detection on updates, and differential privacy. This paper evaluates *undefended* FedAvg against a simple but realistic label-flipping adversary on one client, establishing an empirical baseline for how much "natural dilution" multi-client averaging provides before specialized defenses are added.

E. UNSW-NB15 Family of Datasets

UNSW-NB15 and related CIC-augmented releases provide labeled flow features covering multiple attack categories alongside normal traffic. We use an augmented CIC–UNSW-NB15 tabular release with d = 76 numeric features and C = 10 classes. Exact semantic names of minority classes are less critical for our algorithmic conclusions than the empirical imbalance pattern: one dominant class and several rare classes with near-zero recall under a simple MLP. This imbalance is typical of security telemetry and motivates confusion-matrix reporting.

III. THEORETICAL FRAMEWORK

A. Supervised Multi-Class Risk

Let ■ be a distribution over ■ × ■ with ■ = {0,…,C−1}. A parametric model f_θ: ■ → $■^C$ produces logits; the predicted label is ■ = arg $\max_c [f_\theta(x)]_c$. The population risk under cross-entropy is

$$R(\theta) = ■_{(x,y)\sim■} [\ ■(f_\theta(x), y)\ ] \quad (1)$$

where $■(z, y) = -\log(e^{z_y} / \Sigma_c e^{z_c})$. Given a finite training set $S = \{(x_i, y_i)\}_{i=1}^{n}$, the empirical risk is

$$R■_S(\theta) = (1/n) \Sigma_{i=1}^{n} ■(f_\theta(x_i), y_i). \quad (2)$$

B. Federated Partition of the Empirical Risk

Partition S into N disjoint client datasets S_k with $n_k = |S_k|$ and $\Sigma_k n_k = n$. The global empirical risk decomposes exactly as a mixture of local empirical risks:

$$R■_S(\theta) = \Sigma_{k=1}^{N} (n_k/n) R■_{S_k}(\theta). \quad (3)$$

Equation (3) is the conceptual justification for size-weighted FedAvg: averaging models trained on local risks approximates minimizing a weighted combination of those risks when local updates remain sufficiently close to the broadcast initialization (limited client drift).

C. Local Optimization

In round t, client k receives θ^t and approximately solves

$$\theta_k^{t+1} \approx \arg\min_\theta R■_{S_k}(\theta) \text{ initialized at } \theta^t, \quad (4)$$

using E epochs of minibatch Adam with learning rate η. Let B be the minibatch size. The number of local gradient steps per round is roughly $E \cdot ■n_k/B■$. More local steps increase client drift away from θ^t but also increase local progress per communication round—the classic FedAvg trade-off between computation and communication. In our implementation, the Adam optimizer state is re-initialized each round from the broadcast weights, so momentum does not persist across rounds.

D. FedAvg Aggregation

The server sets

$$\theta^{t+1} = \Sigma_{k=1}^{N} w_k \theta_k^{t+1},\ w_k = n_k / \Sigma_j n_j. \quad (5)$$

If shards are equal-sized, $w_k = 1/N$. Aggregation is performed element-wise over all tensors in the state dictionary (weights and biases of every layer). Because every client shares the same architecture, corresponding parameters are aligned and averaging is well-defined.

E. Communication Cost Model

Let |θ| denote the number of scalar parameters. Each round transmits approximately N|θ| parameters uplink and N|θ| downlink (broadcast counted per client), i.e., Θ(N|θ|) communication per round. Total communication over R rounds scales as Θ(R N |θ|). Increasing E can reduce the R required for a target accuracy in many IID settings, improving communication efficiency—one of FedAvg's original motivations. For our compact MLP, |θ| is only tens of thousands of scalars, making full-model exchanges inexpensive in simulation.

F. MLP Hypothesis Class

Our model is a two-hidden-layer MLP:

$$f_\theta(x) = W_3\,\sigma(W_2\,\sigma(W_1 x + b_1) + b_2) + b_3, \quad (6)$$

with σ = ReLU, hidden width 128, d = 76, and C = 10. Softmax is implicit in the cross-entropy loss. ReLU networks can approximate a wide family of piecewise-linear decision regions; in practice, capacity interacts with class imbalance, often favoring majority-class decision rules unless rare classes are sufficiently represented or reweighted.

G. Class Imbalance and Metrics

Overall accuracy $\mathrm{Acc} = (1/n_{test})\,\Sigma_i\, 1[\blacksquare_i = y_i]$ is dominated by majority-class correctness under imbalance. We therefore also report precision, recall, F1, the full confusion matrix $M \in \blacksquare^{C\times C}$ with $M_{ij} = \#\{i' : y_{i'}=i, \blacksquare_{i'}=j\}$, and one-vs-rest counts:

$$TP_c = M_{cc},\ FP_c = \Sigma_{i\neq c} M_{ic},\ FN_c = \Sigma_{j\neq c} M_{cj},\ TN_c = n_{test} - TP_c - FP_c - FN_c. \quad (8)$$

Per-class recall $TP_c/(TP_c+FN_c)$ is especially informative for rare attacks. High TN_c values are expected in multi-class settings because most examples are not of class c; thus TP/FP/FN trends are usually more actionable than TN magnitude alone.

H. Threat Model: Label Poisoning

We consider a dirty-label adversary controlling client 0 only. Features remain untouched; a fraction $f \in [0,1]$ of local labels are replaced according to a mode m:

- **Target:** $y \leftarrow y_{tgt}$ (we use $y_{tgt} = 0$, the majority class).
- **Random-wrong:** $y \leftarrow$ uniform over $\blacksquare \blacksquare \{y_{true}\}$.
- **Cycle:** $y \leftarrow (y+1) \bmod C$.

The adversary does not modify uploaded weights beyond ordinary training on poisoned labels, and does not control the test distribution. This models a compromised or negligent data pipeline at one silo rather than an optimal model-replacement attack.

I. Why Dilution Occurs under FedAvg

Suppose equal shard sizes and one poisoned client. Even if θ_0^{t+1} is strongly biased, the aggregate is

$$\theta^{t+1} = (1/N)\,\theta_0^{t+1} + (1/N)\,\Sigma_{k=1}^{N-1}\,\theta_k^{t+1}. \quad (9)$$

As N grows, the poisoned update's coefficient 1/N shrinks. This is not a cryptographic defense and fails if many clients collude or if the adversary mounts stronger model poisoning; but it explains our empirical observation that multi-client runs remain near clean accuracy under single-client label flips. Conversely, when N = 1, equation (9) collapses to $\theta^{t+1} = \theta_0^{t+1}$, so poisoning transfers undiluted.

J. Convergence Intuition (IID Case)

When shards are IID draws from the same distribution, local risks are statistically similar. Local Adam steps from a shared initialization tend to move in related directions, so averaging is constructive. Formal FedAvg analyses typically assume smoothness/Lipschitz gradients, bounded stochastic gradient variance, and control of client drift as a function of E and heterogeneity. Our shards are IID-like by construction (random shuffle then cut), placing us near the favorable regime. Non-IID partitions would increase drift and could amplify poisoning or reduce clean accuracy; we leave that to future work.

An intuitive view is that FedAvg performs a form of inexact alternating minimization on the mixture objective (3): each client descends on its term, then the server projects back to a consensus model by averaging. With small E and IID data, the inexactness is mild; with large E or heterogeneous data, consensus quality degrades without algorithmic corrections (e.g., proximal terms).

K. On Targeted Majority-Class Poisoning

Targeting the majority class is a strategically strong dirty-label choice under imbalance. Flipping labels to class 0 increases the empirical prior toward 0 inside the poisoned shard. The resulting local optimum tends to expand the region of feature

space assigned to class 0. After aggregation, especially at N = 1, test points from neighboring classes are more often absorbed into class 0 predictions. Random-wrong flips, by contrast, inject less coherent bias and often degrade performance less at equal f in our sweep.

IV. SYSTEM DESIGN AND IMPLEMENTATION

A. Data Pipeline

Figure 1 summarizes preprocessing. Features and labels are aligned row-wise from Dataset.csv and Label.csv. Non-finite values are replaced by zeros. A stratified split (default 80% train / 20% test) preserves class proportions. A StandardScaler is fit on training features only, then applied to train and test. Training indices are shuffled with a fixed seed and split into N approximately equal shards.

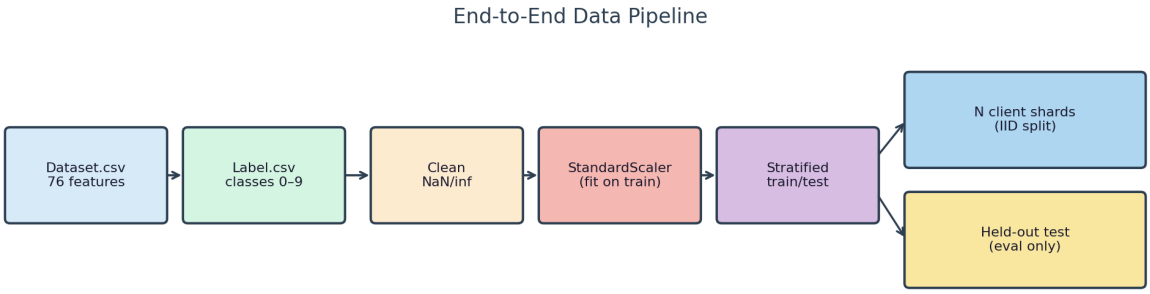


Fig. 1. End-to-end data pipeline from raw CSVs to client shards and a held-out test set.

Note on global scaling. Fitting a scaler on pooled training features is a practical simplification for a single-machine simulation. Strictly decentralized FL might estimate normalization statistics via secure aggregation or use per-client normalization.

B. Model Architecture

Figure 2 shows the shared MLP. Identical architecture across clients is mandatory for element-wise FedAvg. The network is intentionally compact to enable a large configuration sweep on CPU.

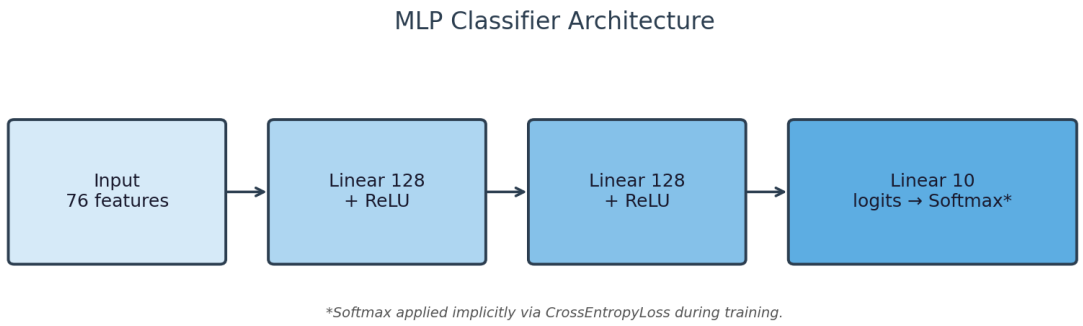


Fig. 2. Shared MLP classifier used by every client.

C. Federated Training Loop

Figure 3 and Algorithm 1 describe one round of training. In code, each client deep-copies the global state dictionary, trains with Adam and cross-entropy, then returns parameters and sample counts for weighted averaging.

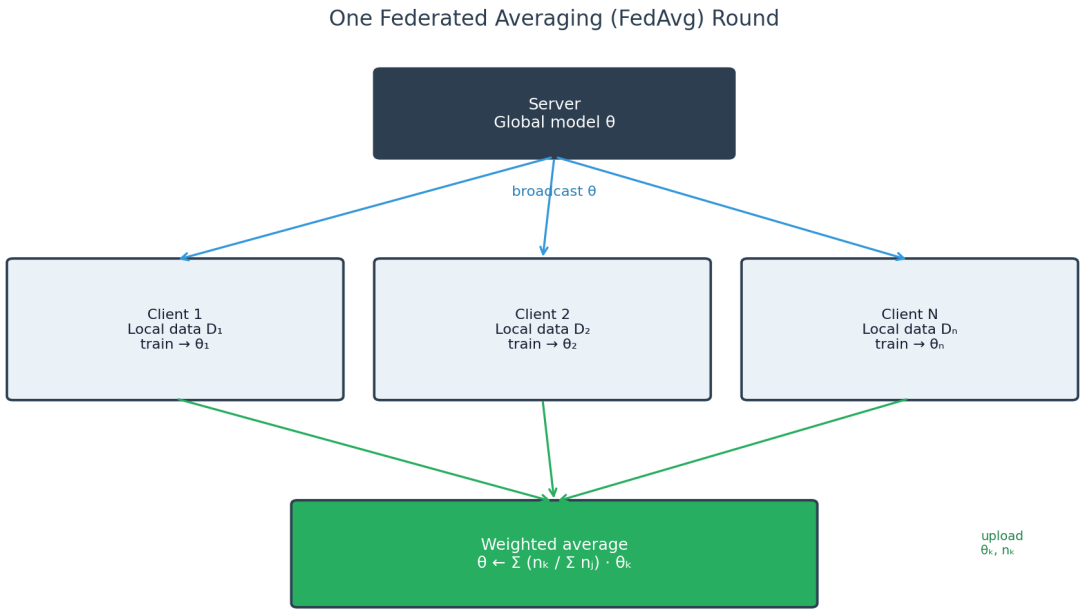


Fig. 3. One FedAvg round: broadcast, local training, and weighted aggregation.

Algorithm 1: FedAvg for intrusion classification (simulation)

1. Initialize global model θ^0.
2. For round t = 0 … R–1:
   a. For each client k = 1…N: set $\theta_k \leftarrow \theta^t$; run E local Adam epochs on S_k; upload (θ_k, n_k).
   b. Aggregate $\theta^{t+1} \leftarrow \Sigma_k (n_k/\Sigma_j n_j)\, \theta_k$.
   c. Evaluate θ^{t+1} on the held-out test set.
3. Return θ^R.

D. Poisoning Hook

Before local training begins, if poisoning is enabled, client 0's label vector is rewritten according to Section III-H (Figure 4). Features and all other clients remain clean. The test set is never poisoned.

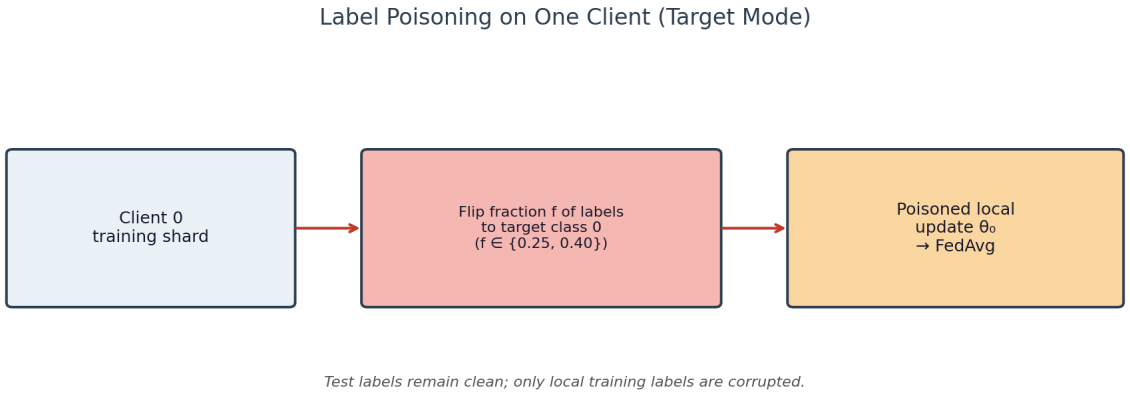


Fig. 4. Targeted label poisoning applied only to client 0 training labels.

E. Software Stack and Configuration

The implementation uses PyTorch for the MLP and optimization; pandas/NumPy for I/O; and scikit-learn for splitting, scaling, confusion matrices, and classification reports. A configuration dataclass exposes clients, rounds, local epochs, batch size, learning rate, device, and poison parameters via CLI flags, enabling reproducible sweeps and scripted

visualization.

V. EXPERIMENTAL SETUP

A. Hardware and Reproducibility

Runs use CPU by default with fixed random seeds for NumPy and PyTorch. This favors reproducibility over wall-clock speed and avoids GPU nondeterminism as a confounder when comparing many nearby configurations.

B. Grid Design

The principal sweep comprises 64 configurations exploring $N \in \{1,2,3,5\}$; global rounds (notably 5 and 10); local epochs $E \in \{1,2\}$; poison fraction $f \in \{0, 0.25, 0.40\}$; and poison mode $\in$ {target, random_wrong} when $f > 0$. Primary metrics are test accuracy, precision, recall, and F1. Selected representative scenarios additionally store full confusion matrices and TP/FP/FN/TN vectors.

C. Descriptive Statistics of the Sweep

Across all 64 runs, accuracy has mean ≈ 0.907, standard deviation ≈ 0.014, minimum ≈ 0.856, and maximum ≈ 0.924. The relatively small standard deviation indicates that most configurations remain in a high-accuracy band; the lower tail is concentrated in heavily poisoned single-client settings. Mean F1 is ≈ 0.901 with a slightly larger spread, consistent with minority-class sensitivity.

D. Evaluation Protocol and Artifacts

The test set is never poisoned and never used for training. Results are exported to CSV; static PNG figures and dashboards accompany the repository. Live dashboard: https://fl-research.vercel.app.

VI. RESULTS

A. Clean Training Behavior

Under clean labels, increasing R and E generally improves accuracy. Single-client clean training can slightly outperform multi-client FedAvg at the same budget because there is no averaging-induced damping; however, multi-client accuracy remains strong (typically > 0.90). Figures 5 and 6 illustrate round and local-epoch effects on clean runs.

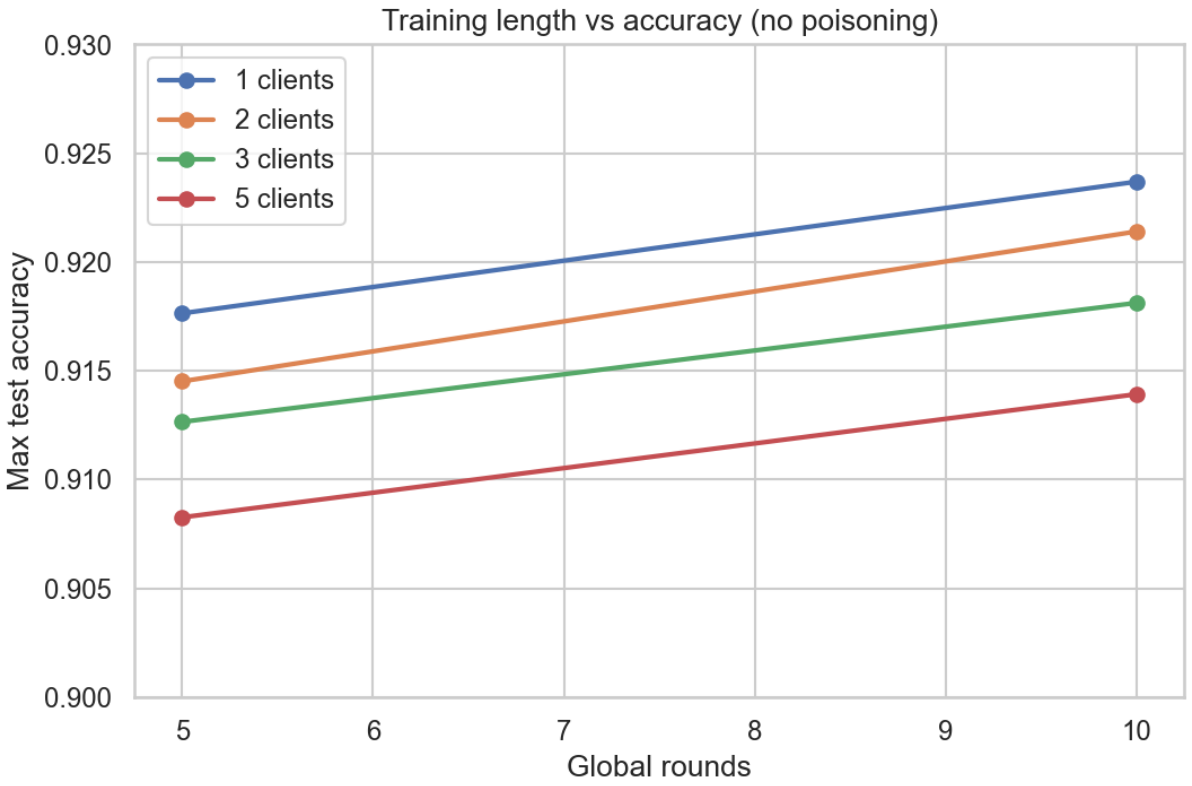


Fig. 5. Effect of global rounds on clean-run accuracy.

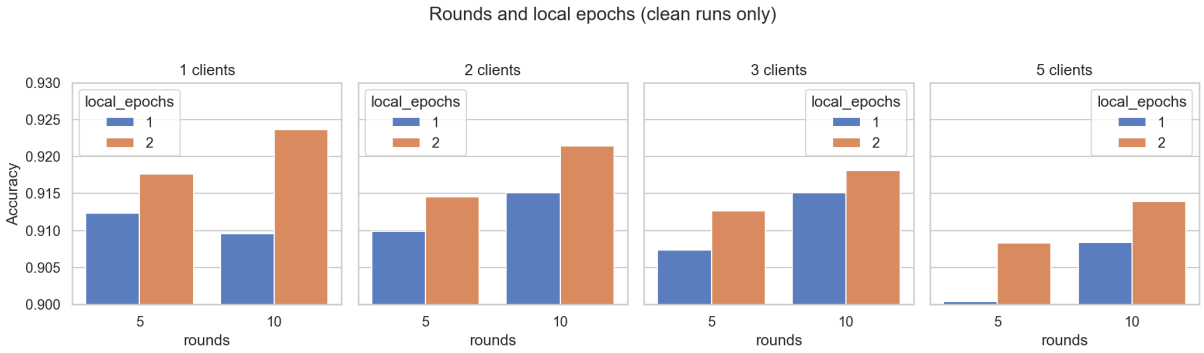


Fig. 6. Local-epoch effects for clean federated runs.

B. Clients and Poison Scenarios

Figure 7 compares best accuracy by client count and scenario. Figure 8 shows a clients-versus-poison heatmap of maximum accuracy. Figure 9 contrasts clean versus 40% target poisoning. Visually, the poison penalty is concentrated at N = 1 and nearly disappears at N = 5.

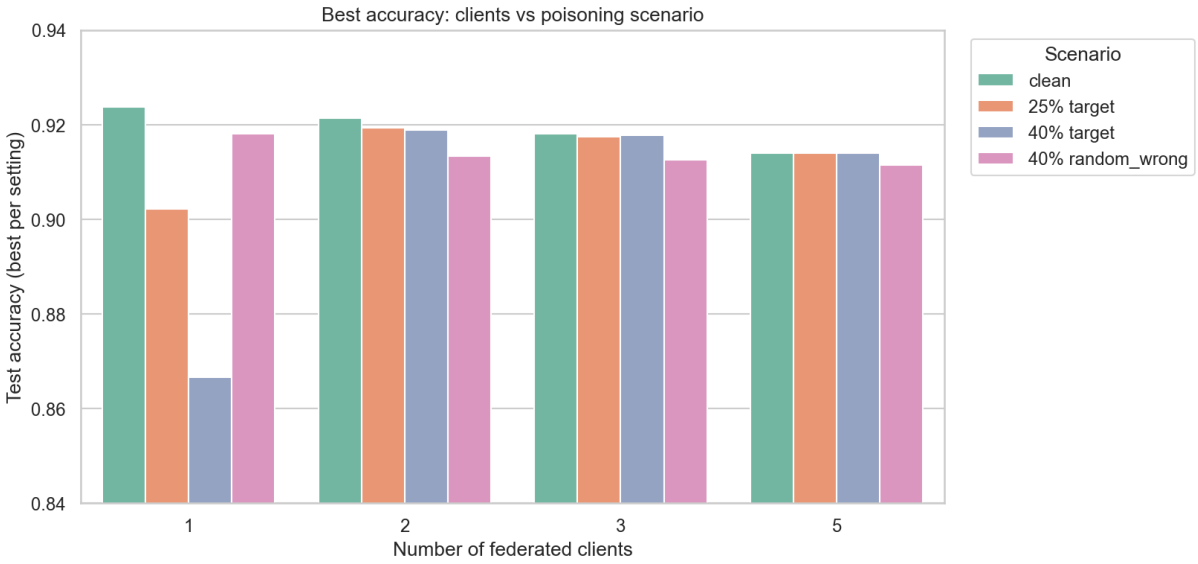


Fig. 7. Best accuracy by client count and poisoning scenario.

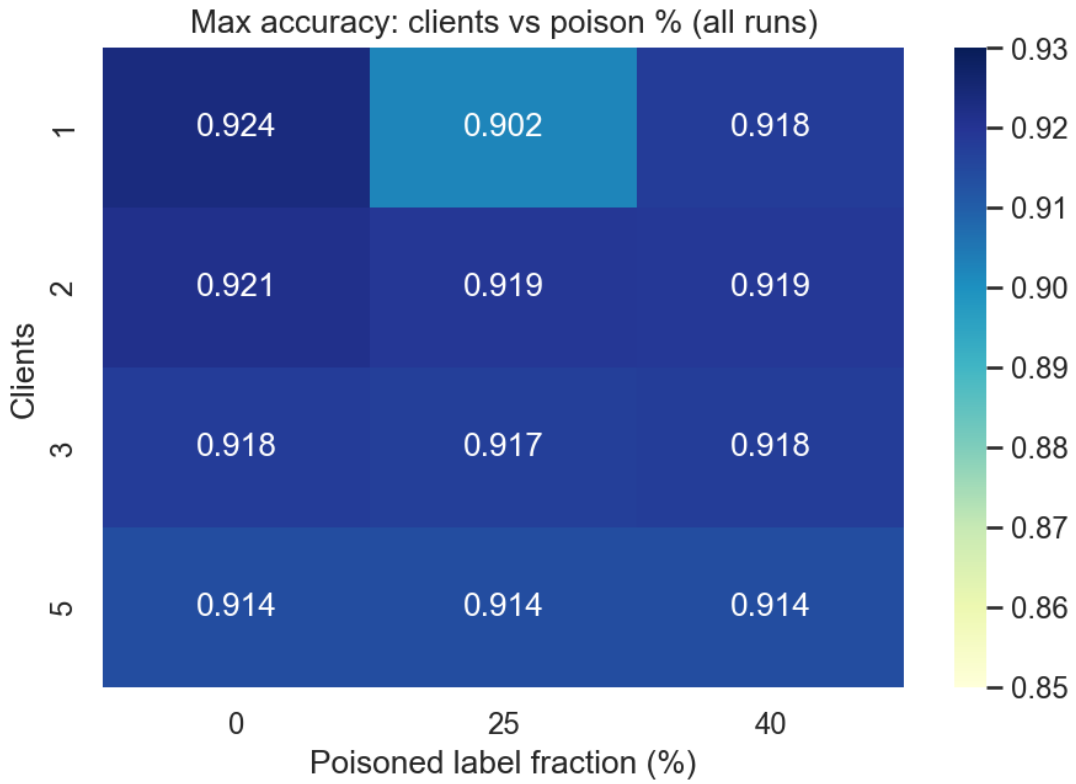


Fig. 8. Max accuracy heatmap over clients versus poison percentage.

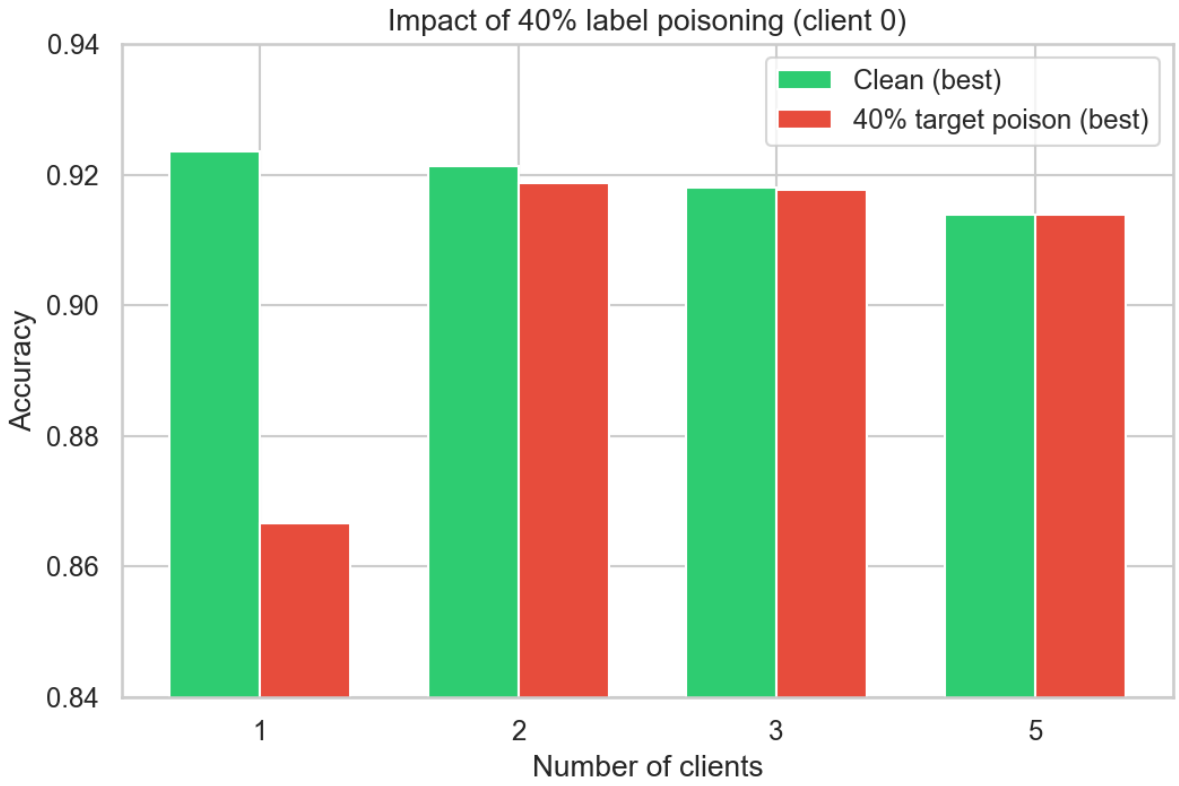


Fig. 9. Clean versus 40% target poisoning (best accuracy per client count).

C. Best Configurations

Table I lists best observed metrics by condition. The most damaging cell is N = 1 with 40% target poison (accuracy ≈ 0.867 in the best-of-condition table; some individual runs fall near 0.856). By contrast, N = 5 with 40% target poison remains essentially tied with the clean five-client best (≈ 0.914).

TABLE I. Best test accuracy by client count and poison setting

Clients	Poison	Mode	Acc.	F1
1	0%	—	0.9237	0.9212
1	25%	target	0.9022	0.8881
1	40%	target	0.8666	0.8329
1	40%	random	0.9180	0.9127
2	0%	—	0.9214	0.9177
2	40%	target	0.9188	0.9132
3	0%	—	0.9181	0.9152
3	40%	target	0.9178	0.9135
5	0%	—	0.9139	0.9119
5	40%	target	0.9139	0.9117

D. Mean Accuracy by Clients and Poison Fraction

Table II reports mean accuracy aggregated over rounds/epochs/modes in the sweep. The drop from f = 0 to f = 0.4 is large for N = 1 (≈ 0.916 → 0.887 mean) and small for N = 5 (≈ 0.908 → 0.905). Intermediate N values interpolate smoothly, supporting the 1/N dilution intuition of Section III-I.

TABLE II. Mean test accuracy by clients and poison fraction

Clients	f = 0	f = 0.25	f = 0.40
1	0.9158	0.8981	0.8873
2	0.9152	0.9141	0.9106
3	0.9133	0.9127	0.9082
5	0.9077	0.9076	0.9051

E. Target versus Random-Wrong Poisoning

Target poisoning toward class 0 is typically more harmful than random-wrong poisoning at the same fraction in low-N regimes. Random-wrong noise is less coherently biased; target poisoning consistently teaches a majority-class prior. This distinction appears both in aggregate metrics and in confusion structure.

F. Confusion Matrices: Clean Multi-Client

Figures 10–11 show five-client clean versus poisoned matrices. They are nearly alike: FedAvg dilution keeps decision boundaries stable. Class 0 dominates correct predictions; rare classes remain difficult; class 5 absorbs many off-diagonal errors even in clean training, indicating a structural confusion pattern of the representation/model rather than an artifact of poisoning alone.

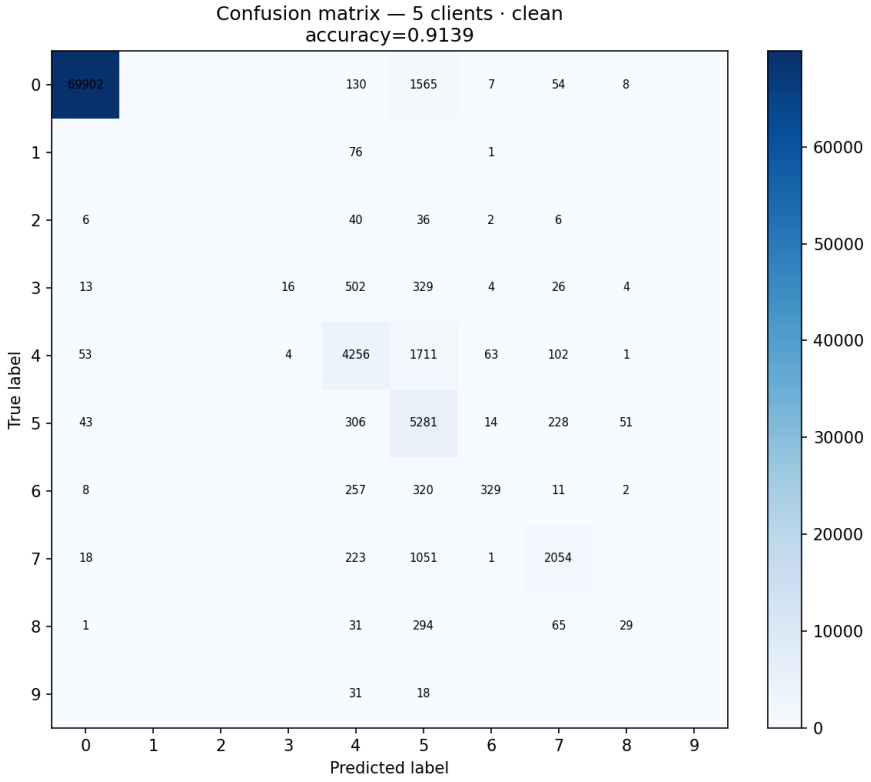


Fig. 10. Confusion matrix: 5 clients, clean training.

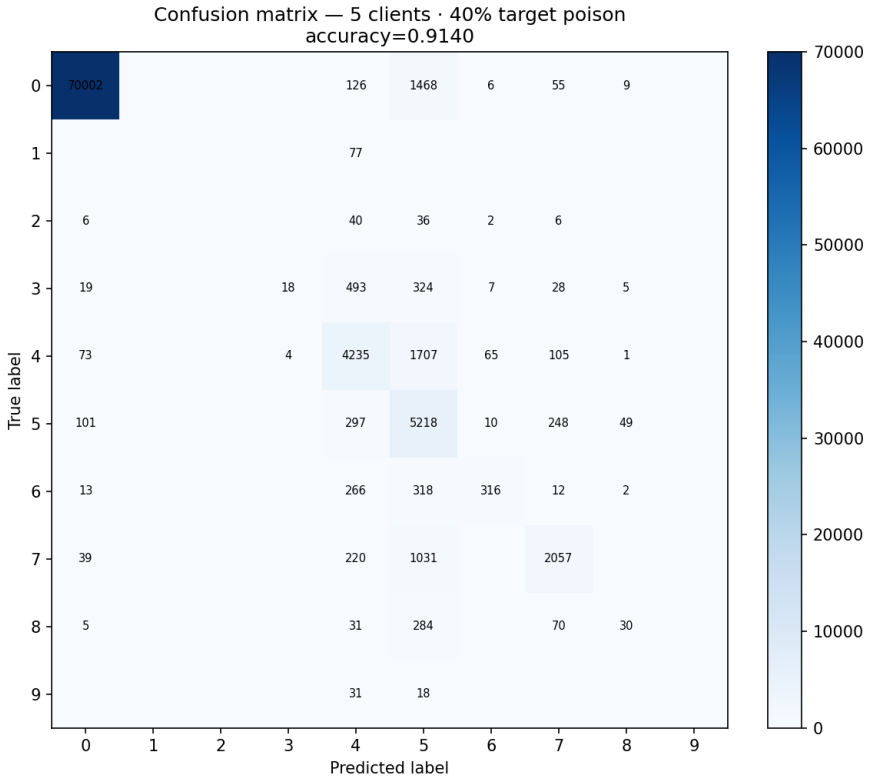


Fig. 11. Confusion matrix: 5 clients, 40% target poison on client 0.

G. Confusion Matrices: Single-Client Fragility

Figures 12–13 show the single-client contrast. After 40% target poisoning, substantial mass moves into predicted class 0 from true classes 4, 5, 6, and 7. Class 0 recall can remain high or even increase, while other classes suffer. Figures 14–15 make the same point via TP/FP/FN/TN bars: class 0 FP rises; FN rises for several non-zero classes. This is the empirical signature of majority-class targeted label noise under undiluted training.

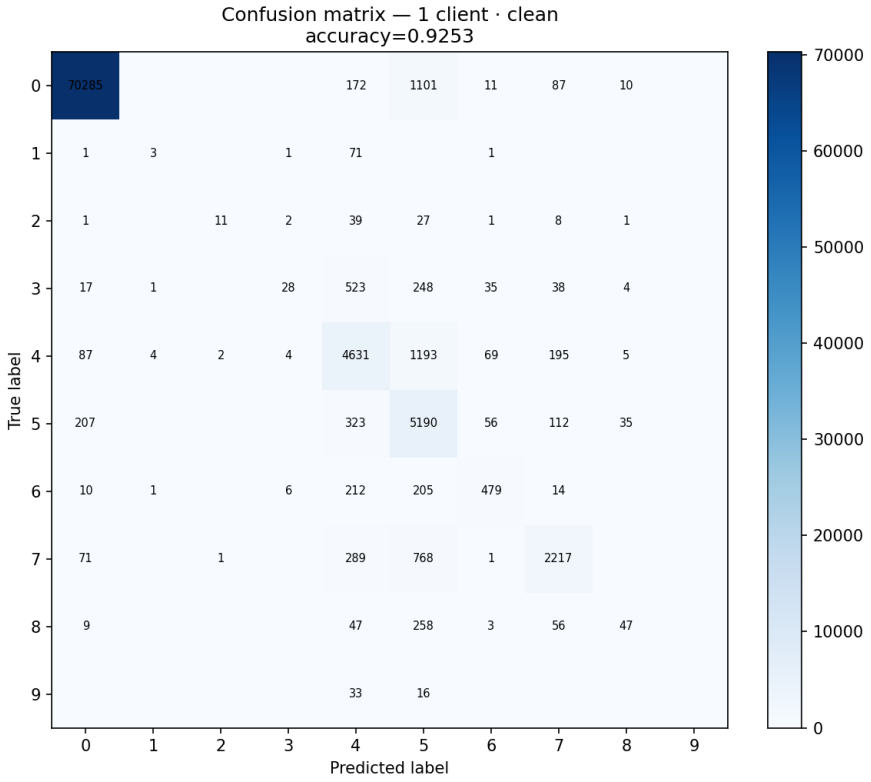


Fig. 12. Confusion matrix: 1 client, clean training.

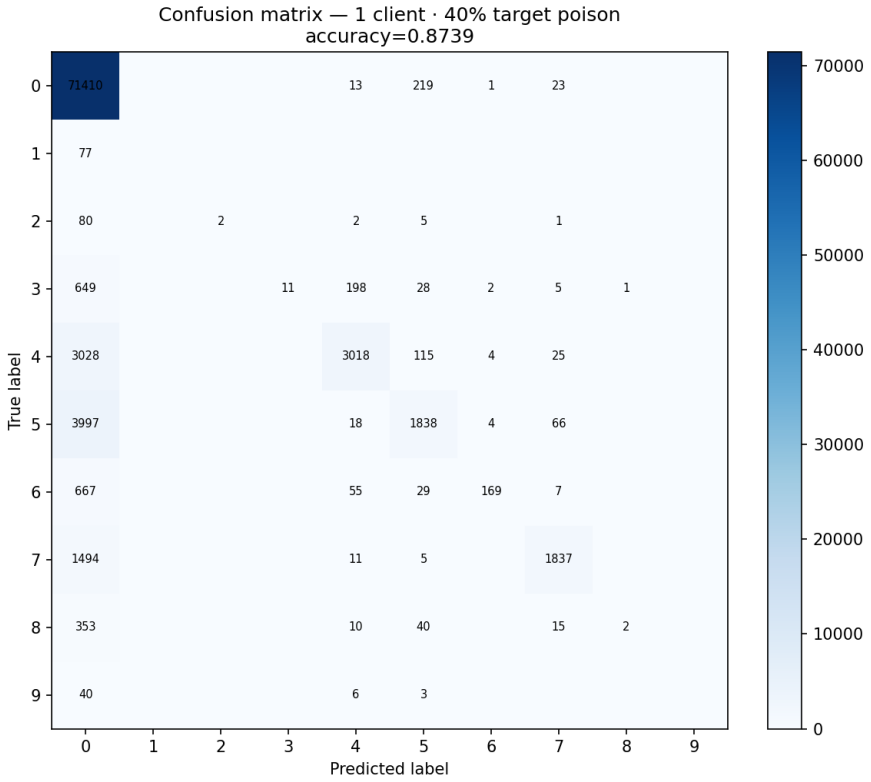


Fig. 13. Confusion matrix: 1 client, 40% target poison.

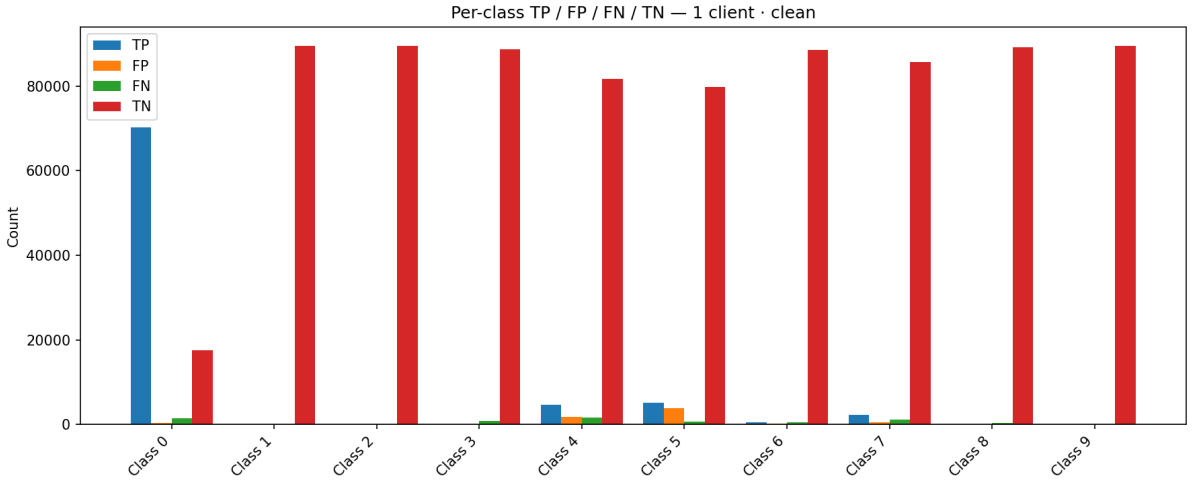


Fig. 14. Per-class TP/FP/FN/TN: 1 client, clean.

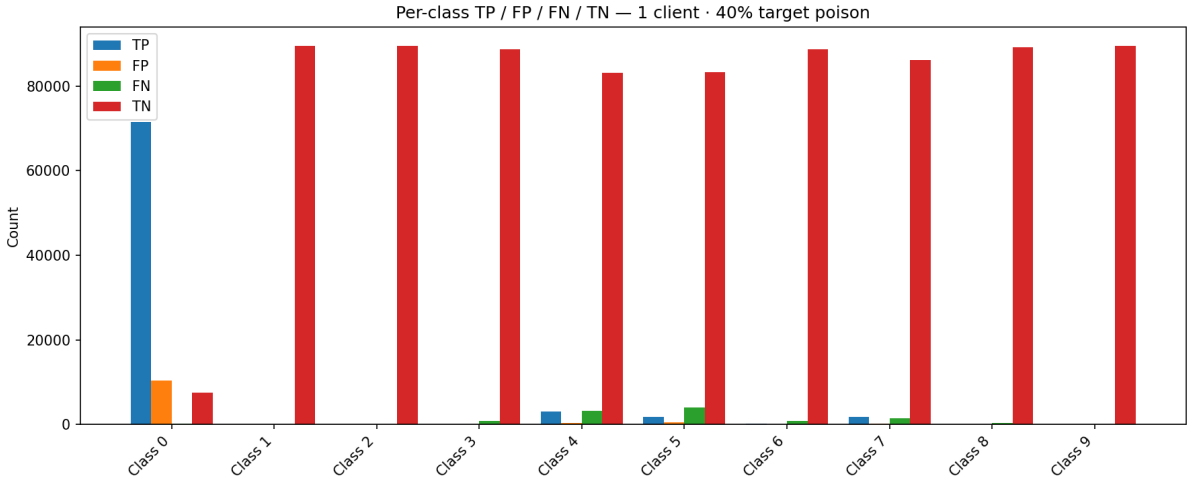


Fig. 15. Per-class TP/FP/FN/TN: 1 client, 40% target poison.

H. Security Interpretation of Errors

From a SOC analyst's perspective, elevated class 0 false positives after target poisoning mean attack flows are more often declared "majority/benign-like" (depending on label semantics), i.e., missed detections. Overall accuracy may still look acceptable because class 0 is frequent. This is why confusion-level reporting is mandatory for security ML evaluations and why Table I's accuracy gaps understate operational harm at N = 1.

VII. DISCUSSION

A. Implications for Deploying FL in IDS

1. Prefer multiple independent data silos when possible; averaging provides baseline robustness to one bad silo under label noise.
2. Prefer adequate R and E; under-trained models are both less accurate and potentially less stable under noise.
3. Monitor per-class recall, not only accuracy, especially for rare attack types.
4. Treat single-client "federated" training as essentially centralized: it inherits no dilution benefit.
5. Use dashboards and confusion audits in experiment tracking so regressions in minority classes are visible.

B. Relationship to Robust FL Theory

Byzantine-robust aggregators were designed for stronger adversaries than our label-flip setting. Our results suggest that for a *single* weakly malicious client under IID equal shards, classical FedAvg already limits damage. That should not be read as a substitute for robust aggregation when clients can send arbitrary updates, when many clients are malicious, or when data are highly non-IID. Instead, dilution is a baseline phenomenon on top of which stronger defenses can be layered.

C. What This Study Does Not Claim

We do not claim FedAvg is secure against adaptive model-poisoning or colluding clients. We do not claim MLP is the optimal IDS architecture. We do not claim IID shards represent real institutional heterogeneity. We do not claim that accuracy near 0.91 is sufficient for production detection without thresholding, ensembling, or human triage.

D. Limitations

- **IID simulation:** random shards underestimate heterogeneity-induced drift.
- **Global scaler:** pooled normalization is a centralized shortcut.
- **Single architecture:** no comparison to trees, CNNs on raw packets, or sequence models.
- **Undefended aggregation:** no median/Krum/DP baselines in the sweep.
- **Label semantics:** class indices are used without exhaustive attack taxonomy narration.
- **No systems noise:** no stragglers, partial participation, or compression artifacts.

E. Threats to Validity

Random seeds, hyperparameter choices (Adam learning rate, batch size), and the particular augmented dataset release could shift absolute numbers. Relative conclusions—dilution with N, severity of target poison at $N = 1$, need for confusion analysis—were consistent across the sweep's internal variation, which increases confidence in the qualitative story.

F. Future Work

1. Non-IID label and feature skew across clients.
2. Robust aggregators and anomaly detection on updates.
3. Differentially private FL for formal privacy guarantees.
4. Cost-sensitive losses or resampling for rare classes.
5. Multi-poisoner and model-replacement attacks.
6. Deployment in a real FL framework with networked workers.
7. Richer backbone models and calibrated probability outputs for SOC workflows.

VIII. CONCLUSION

We presented an expanded theoretical and experimental study of FedAvg for multi-class traffic classification on CIC–UNSW-NB15. Starting from the global empirical risk and its client-wise decomposition, we formalized local optimization, size-weighted aggregation, communication scaling, class-imbalance metrics, and a single-client label-poisoning adversary. Empirically, among 64 configurations, clean training benefits from more rounds and local epochs; targeted poisoning is highly damaging when $N = 1$ and largely diluted when $N \geq 2$; and confusion matrices reveal majority-class bias that accuracy can mask. These findings support a practical message: federated collaboration with multiple honest clients, sufficient training, and per-class monitoring forms a stronger baseline than accuracy-only evaluation of single-silo training under label noise.

ACKNOWLEDGMENT

The author thanks the maintainers of CIC–UNSW-NB15 related public datasets and the developers of PyTorch, NumPy, pandas, and scikit-learn.

REFERENCES

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in Proc. AISTATS, 2017, pp. 1273–1282.

[2] P. Kairouz et al., “Advances and open problems in federated learning,” Found. Trends Mach. Learn., vol. 14, no. 1–2, pp. 1–210, 2021.

[3] S. A. Rahman, H. Tout, C. Talhi, and A. Mourad, “Internet of Things intrusion detection: Centralized, on-device, or federated learning?” IEEE Network, vol. 34, no. 6, pp. 310–317, 2020.

[4] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, “How to backdoor federated learning,” in Proc. AISTATS, 2020, pp. 2938–2948.

[5] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, “Data poisoning attacks against federated learning systems,” in Proc. ESORICS, 2020, pp. 480–501.

[6] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, “Federated learning: Challenges, methods, and future directions,” IEEE Signal Process. Mag., vol. 37, no. 3, pp. 50–60, 2020.

[7] K. Bonawitz et al., “Towards federated learning at scale: System design,” in Proc. MLSys, 2019.

[8] N. Moustafa and J. Slay, “UNSW-NB15: A comprehensive data set for network intrusion detection systems,” in Proc. MilCIS, 2015, pp. 1–6.

[9] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, “Machine learning with adversaries: Byzantine tolerant gradient descent,” in Proc. NeurIPS, 2017.

[10] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, “Byzantine-robust distributed learning: Towards optimal statistical rates,” in Proc. ICML, 2018.